

DocPulse

Project Documentation

Student: [Your Name Here]

Course: B.Tech CSE - Semester II

Date: 12 June 2026

Problem Statement

Organizations hold large collections of documents. Users expect fast search results; administrators need indexing

Objectives

- Demonstrate practical use of Trie, Stack, Queue, Heap, Hash-set, and Graph data structures.
- Provide a CLI to add documents, search by path prefix, queue/index documents, detect duplicates, rank results,
- Explain design choices and time/space complexity for educational use.

Design / Architecture

Modules (each in header+implementation files) and responsibilities:

- Document store (`Document.h` / implemented in `main.cpp``): holds document metadata (id, title, path, content, s
- Trie (`Trie.h/.cpp``): fast prefix-based lookup for file paths.
- ConfigManager (`ConfigManager.h/.cpp``): stack-based undoable configuration management.
- IndexQueue (`IndexQueue.h/.cpp``): FIFO queue for indexing pipeline.
- FileIntegrity (`FileIntegrity.h/.cpp``): signature generation and duplicate detection using a hash-set.
- RelevanceSorter (`RelevanceSorter.h/.cpp``): max-heap to return top relevant documents.
- KnowledgeGraph (`KnowledgeGraph.h/.cpp``): adjacency-list graph of citations with BFS shortest-path.
- SpaceManager (`SpaceManager.h/.cpp``): greedy max-heap suggesting compression order.

The ``main.cpp`` wires these modules together and provides a menu-driven CLI for user interaction.

Algorithm Description (summary)

CODE_EXPLANATION for DocPulse

This document explains every source and header file in the DocPulse project in plain, layman-friendly language. It

1) Document.h

- Purpose: defines the Document "record" used throughout the program. Think of Document as a folder card that t

Contents explained:

- ``#pragma once`` : Stops this file from being accidentally included more than once.
- ``#include <string>`` and ``using std::string;`` : Lets the code use the string type to store text.
- The ``Document`` class lists properties:
 - ``int id;`` : a unique number to identify the document.
 - ``string title;`` : human-readable title.
 - ``string path;`` : file path like ``/eng/specA.txt``.
 - ``string content;`` : the actual text (simplified for this project).
 - ``string signature;`` : a short, fixed-size string generated from title+path+content to check uniqueness.
 - ``int relevanceScore;`` : a score used to rank how relevant a document is for search results.
 - ``int fileSize;`` : approximate file size in KB; used by SpaceManager.
- The constructors and ``generateSignature()`` are declared here; their implementation lives in ``main.cpp`` so the cla

Why it matters: ``Document`` holds all metadata in one object so every module (indexer, trie, graph, etc.) can share

2) ConfigManager.h / ConfigManager.cpp

- Purpose: stores and manages search engine settings. Allows "undo" so the latest change can be rolled back.

Key ideas in ``ConfigManager.h``:

- It keeps a ``std::stack<string>`` named ``history`` which stores old settings.
- ``currentConfig`` is the currently active configuration string.
- Public methods: ``changeConfig()``, ``undo()``, ``getCurrentConfig()``.

What the `` .cpp`` does (behavior explained):

- Constructor sets ``currentConfig`` to "default".
- ``changeConfig(newConfig)``:
 - pushes the current setting onto the stack so it can be restored later.
 - updates ``currentConfig`` to the new value.
- ``undo()``:
 - If the stack is empty, nothing to undo.
 - Otherwise, pop the top of stack and set it as ``currentConfig``.
- ``getCurrentConfig()`` returns the currently active configuration.

Layman explanation: think of the stack as an "undo stack" in a text editor: every time you change a setting, we re

3) IndexQueue.h / IndexQueue.cpp

- Purpose: model a pipeline where uploaded documents are queued for indexing in the order they arrive (first-in-first-out).

Key parts:

- `std::queue<Document> q;` holds documents.
- `enqueue(doc)` pushes a document to the back.
- `dequeue(outDoc)` removes the front document and returns it by reference. Returns false if the queue is empty.
- `displayPending()` prints how many are pending.

Complexity Analysis

(n = number of documents, L = length of a key/prefix, V/E = graph nodes/edges)

- Trie insert/search: Time $O(L)$, Space $O(\text{sum of characters stored in nodes})$.
- Stack: $O(1)$ time per operation, $O(k)$ space for k saved states.
- Queue: $O(1)$ time per operation, $O(m)$ space for m queued items.
- Signature generation & lookup: average $O(1)$ time, $O(n)$ space for stored signatures.
- Priority queue operations: $O(\log n)$ for insert/pop.
- Graph (BFS): $O(V + E)$ time, $O(V)$ extra space for visited/parent bookkeeping.

Limitations: the Trie children array increases memory usage; `std::hash` is not collision-resistant.

Menu

```
$ ./docpulse
DocPulse - Menu
1) Add Document
2) Search File Path Prefix
3) Change Search Configuration
4) Undo Configuration
5) Upload Document for Indexing
6) Process Next Document
7) Verify Duplicate Document
8) Rank Documents by Relevance
```

Search Results

```
$ ./docpulse
Enter choice: 2
Enter prefix: /eng
Matches:
- ID: 1, Title: Design Spec A, Path: /eng/specA.txt, Relevance: 85
- ID: 2, Title: Engineering Notes, Path: /eng/notes.txt, Relevance: 70
```

Ranked Results

```
$ ./docpulse
Enter choice: 8
Ranked documents (highest relevance first):
1) ID: 1 Title: Design Spec A Relevance: 85
2) ID: 2 Title: Engineering Notes Relevance: 70
```

Results and Findings

See README.md for detailed observations and limitations. The project demonstrates trie-based prefix search, sta

Conclusion

DocPulse is an educational demonstration of common data structures applied to document search.